

Problem A. Moniphant Sleep

For every Moniphant, the sequence of operations applied to it can be regarded as one state transformation. A range update therefore does not need to be pushed to each position immediately; it is enough to compose the same transformation into the lazy tag of the corresponding segment tree nodes.

The main difficulty is how to merge two tags. A tag can be normalized into the following short form: first move down several times, then move up several times, then possibly execute operation 4, then again move down and up, then possibly execute operation 3, and finally move up several times. When two normalized tags are composed, the only nontrivial case is whether the operation 4 in the later tag can return to a mark produced by operation 3 in the earlier tag. After this case distinction, the composition can again be written in the same normalized form.

Maintain a lazy segment tree over the indices. Each node stores this normalized transformation as its lazy value. For a range operation, compose the new elementary transformation after the old one. For a point query, collect all transformations on the path from the root to the leaf, compose them in chronological order, and apply the result to the initial depth.

The correctness follows from the invariant that the lazy tag of every segment tree node is exactly the chronological composition of all operations that fully covered this interval and have not been pushed below it. Since normalized tags preserve the behavior of the original operation sequence, the point query obtains the same final depth as direct simulation.

Time complexity: $O(n + q \log n)$.

Problem B. Click the Circle

Simulate all pairs of objects geometrically. Split the life of each sliding note into the three time intervals $[l - d, l]$, $[l, r]$, and $[r, r + d]$. On these intervals the center is respectively stationary, moving on one segment, and stationary again. The frame of a slide is a fixed segment existing during its active interval.

After this decomposition, every test that matters is one of the following:

- moving point versus moving point; fixing one point turns it into a point-to-segment distance test,
- moving point versus fixed segment; this turns into a segment-to-segment distance test,
- fixed segment versus fixed segment; this is also a segment-to-segment distance test.

Only pairs whose time intervals intersect need to be checked. On the common time interval, compute the endpoints of the moving trajectory and test whether the minimum distance is at most $2r$.

Thus, for every pair of original objects, inspect all relevant circle-circle, circle-frame, and frame-frame component pairs. The special rule that a slide's circle and frame are considered different objects is handled naturally by treating them as separate components.

Time complexity: $O(n^2)$.

Problem C. Tree

Use heavy-light decomposition together with a dynamic-DP style data structure. For each vertex x , maintain the information contributed by its light subtrees in multisets. One light subtree contributes its internal answer and its longest downward chain. Such a chain is summarized by its pair of colors (a, c) and its length.

If a good path has LCA x and both ends lie in light subtrees of x , then its vertex color must be a_x . For each vertex color a , maintain, over all edge colors c , the sum of the two longest chains of type (a, c) . This can be updated by a constant number of multiset operations whenever one light-subtree contribution changes.

Now consider paths whose LCA lies on a heavy chain. Put the color of a heavy edge onto the lower endpoint. For two adjacent vertices x and y on a heavy chain, the path information can be merged through three kinds of pieces:

- the top piece, which requires $a_x = a_y$ and may attach the best light-chain of type (a_y, c_y) ,
- the middle piece, which requires $(a_x, c_x) = (a_y, c_y)$ and contributes the edge weight w_x ,
- the bottom piece, which may attach the best light-chain of type (a_x, c_x) and then add w_x .

Store these transition data in a segment tree for each heavy chain, so that chain intervals can be merged and queried quickly.

When the color of a vertex changes, first remove the contributions of all heavy chains on the path to the root, update the local information of the changed vertex and its parent, and then climb the heavy chains again while reinserting and recomputing the affected data.

Every good path is uniquely decomposed into light-subtree chains and a heavy-chain middle part at its LCA. The data above enumerates exactly these decompositions, and every candidate maintained by it satisfies the required vertex-color and edge-color equalities.

Time complexity: $O((n + q \log n) \log n)$.

Problem D. Alice and Bob

Let $k = p_1$. Alice loses if and only if the first k values are all at least k . Indeed, if this condition holds, then no matter how Alice permutes the first k positions, Bob can put k back into the first position. Alice will eventually be forced to operate with the same first value again. Conversely, if the condition does not hold, Alice can sort the first k positions and give Bob the analogous losing structure.

Therefore the counting problem is reduced to enumerating $k = p_1$. For fixed k , the first position is k , and the remaining $k - 1$ positions among the prefix must be filled by values chosen from $\{k + 1, \dots, n\}$. The remaining values can be placed arbitrarily. Hence the number of Bob-winning permutations is

$$\sum_{k=1}^{\lfloor (n+1)/2 \rfloor} \binom{n-k}{k-1} (k-1)!(n-k)! \pmod{998244353}.$$

The formula can be evaluated by precomputing factorials and inverse factorials, or by updating consecutive terms directly.

Time complexity: $O(n)$.

Problem E. Neighbourhood

Build the centroid decomposition of the tree. For each centroid c , the standard centroid-decomposition query reduces the original question to counting, among the vertices stored at c , how many have current distance at most a given bound. The duplicated vertices from the same child component are subtracted using an auxiliary structure.

Now consider an edge-weight update by Δ . With respect to any fixed centroid, exactly one side of that original tree edge has all its distances to the centroid changed by Δ , while the other side is unchanged. If we order the vertices of every centroid component by a DFS order, this affected side becomes a contiguous interval. Thus the data structure needed at each centroid is:

- add a value to all depths in an interval,
- count how many current depths are at most d .

This can be implemented by square-root decomposition: each block has a lazy addition value and a sorted list of its explicit depths. Whole blocks are updated lazily, and boundary blocks are rebuilt after direct modification. Counting is done by binary search inside the sorted blocks.

Although one operation touches $O(\log n)$ centroid ancestors, the component sizes shrink by a factor of two along the centroid tree. Therefore the total work has the same order as the sequence data-structure cost, up to implementation-dependent logarithmic factors.

Time complexity: $O(\sqrt{n} \log n)$ to $O(\sqrt{n})$ per operation, depending on the implementation.

Problem F. Cover

Root the tree. Let $dp(x)$ be the best value achievable using only paths completely inside the subtree of x . Let $g(x)$ denote the best contribution from the part of the parent's subtree after the subtree of x and the edge to the parent are removed. Paths are processed at their LCA.

For a path whose LCA is x , its value consists of its own weight, the dp contributions of the two endpoint subtrees, and the sum of the relevant g values along the two chains to the endpoints. The chain sums of g can be maintained by range additions and queries on a segment tree over an Euler order or an HLD order.

At a fixed vertex x , several selected paths with LCA x may conflict only on the first edges from x to its children: each such child edge may be covered by at most one chosen path. Thus the local choice at x is a maximum-weight matching problem on a graph whose vertices are the incident child directions of x . Since the maximum degree is k , this can be solved by subset DP in 2^k , or by any small-graph matching routine with polynomial dependence on k .

Every path is considered exactly once at its LCA. The matching condition captures all conflicts at that LCA, while the subtree DP values already encode optimal choices below the child edges. Combining these local optima bottom-up gives the global optimum.

Time complexity: $O(n \log n + (n + m)2^k)$, or $O(n \log n + (n + m) \text{poly}(k))$.

Problem G. Teleport

The shortest distance is found by BFS because every move costs one. The only issue is that from one cell there may be many teleport targets, and enumerating them repeatedly would be too slow.

Use disjoint-set “next pointers” to maintain the unvisited cells in every family of possible teleport targets. For this problem, repeated applications of the teleport function put the targets on a small number of simple lines, so each BFS state can ask the corresponding DSU structures for the next unvisited candidate in the reachable interval. Once a cell is visited, delete it from all relevant DSU structures by linking it to the next candidate.

Ordinary four-neighbor grid moves are processed normally. For teleports, already visited cells are skipped by the DSU, while every newly discovered cell is inserted into the BFS queue exactly once. Hence the DSU optimization preserves the BFS order and only avoids redundant scans.

Time complexity: $O(n^2 \alpha(n))$, usually written as $O(n^2)$.

Problem H. Function

Rewrite the recurrence by using the current limit as the state. Define $F(t)$ to be the value corresponding to the original state with remaining upper bound t . Then

$$F(0) = 0, \quad F(t) = 1 + \sum_{k=2}^{\min(20210926, t)} F\left(\left\lfloor \frac{t}{k} \right\rfloor\right).$$

The values $\lfloor t/k \rfloor$ take only $O(\sqrt{t})$ distinct values. For a fixed left endpoint l , let $v = \lfloor t/l \rfloor$ and $r = \lfloor t/v \rfloor$; then all $k \in [l, r]$ contribute the same value $F(v)$. This gives a memoized divide-and-conquer recursion with floor-division grouping.

An alternative interpretation is that $F(t)$ counts multiplicative sequences whose factors lie from 2 to 20210926 and whose product is at most t . If the upper bound on the factor is ignored this is easier, and because the bound is very large, inclusion-exclusion over the forbidden large factors leads to a faster variant.

Time complexity: $O(n^{3/4})$ with the grouped recursion; $O(n^{2/3+\varepsilon})$ is possible with the multiplicative interpretation.

Problem I. Digit

Let $E(x)$ be the expected number of remaining turns starting from x . If $x > N$, then $E(x) = 0$. Suppose x has L decimal digits and the digit d appears cnt_d times. Then

$$E(x) = 1 + \sum_{d=0}^9 \frac{\text{cnt}_d}{L} E(x(d+1)).$$

The term $d = 0$ is a self-loop. Move it to the left side:

$$E(x) = \frac{L + \sum_{d=1}^9 \text{cnt}_d E(x(d+1))}{L - \text{cnt}_0}.$$

All arithmetic is performed modulo 998244353, so the denominator is inverted modulo this prime.

For $d \geq 1$, the multiplier $d+1$ has only the prime factors 2, 3, 5, 7. Hence every useful state has the form

$$x_0 2^a 3^b 5^c 7^e \leq N,$$

where x_0 is the starting value. There are very few such states. Compute $E(x)$ by memoized DFS; all non-self transitions go to larger numbers, so after removing the self-loop the dependency graph is acyclic.

Time complexity: $O(\log^4 N)$ states in theory, and about $\log^4 N / 250$ states in practice.

Problem J. Leaves

For every node u and every possible number of swaps j , let $dp(u, j)$ be the lexicographically smallest leaf sequence obtainable from the subtree of u using exactly j swaps. If u is a leaf, only $dp(u, 0)$ exists and it is the one-element sequence containing its label.

If u has children L and R , there are two cases. Without swapping at u , combine

$$dp(L, t) + dp(R, j - t).$$

With swapping at u , combine

$$dp(R, t) + dp(L, j - 1 - t).$$

Try all feasible t and keep the lexicographically smallest resulting sequence. Since $n \leq 1000$, storing the actual sequences and comparing them directly is acceptable; the number of leaves is only about half of n .

The recurrence is complete because the final order at every internal node is determined solely by whether that node is swapped, and the two child subproblems are independent after the number of swaps assigned to them is fixed.

Time complexity: $O(n^3)$.

Problem K. water235

The answer is

$$\left\lceil \frac{N + M}{2} \right\rceil.$$

For the lower bound, look at the perimeter of the water region. When a new empty cell becomes water, it has at least two water neighbors, so the total perimeter of the water region does not increase. At the end the whole rectangle has perimeter $2(N + M)$, while k initially watered cells have perimeter at most $4k$. Therefore $4k \geq 2(N + M)$.

For construction, take the boundary path

$$(1, 1), (1, 2), \dots, (1, M), (2, M), \dots, (N, M).$$

Place water on every other cell of this path. In the parity case where this is short by one, also place water on the last cell. The initially skipped cells on the boundary path are filled first, each having two water neighbors, and then the interior is forced one diagonal layer at a time.

The same construction also covers the one-row or one-column cases.

Time complexity: $O(NM)$, because the grid itself has to be output.

Problem L. Square

Arrange the positive integers in triangular rows:

$$1 / 2, 3 / 4, 5, 6 / \dots$$

If x is in row r and column c , then

$$x + \lfloor \sqrt{2x} + 1.5 \rfloor$$

is exactly the cell in row $r + 1$, column $c + 1$. The operation $x \mapsto x - 1$ moves to the immediately previous integer in this triangular order.

If $x \geq y$, the answer is clearly $x - y$. Otherwise, solve the problem backward from y to x . The reverse operations are $y \mapsto y + 1$ and moving one cell to the upper-left. Whenever $y > x$, greedily move upper-left as much as possible; if the current cell is in the first column, do one $+1$ move first and then continue. This greedy choice is optimal because an upper-left move decreases the value by more than a single $+1$ move can compensate, and delaying it never creates a better position.

To implement this in constant time, find the row r and column c of y by triangular numbers. A block of several upper-left moves decreases y by a quadratic expression, so the maximum feasible number of moves is obtained by an integer square root and then corrected by a few checks.

Time complexity: $O(1)$ per test case.

Problem M. Delete the Tree

Build the centroid decomposition tree of the given tree. Assign each original vertex its depth in the centroid tree. Output all vertices of the deepest depth in one operation, then all vertices of the next depth, and so on until the root depth is deleted.

This uses few enough operations: removing a centroid leaves components of size at most half the previous size, so the height of the centroid tree is at most $\lceil \log_2(n + 1) \rceil$, which is at most 10 under the constraints.

It remains to justify that each printed layer is an independent set in the current graph. Vertices of the same centroid depth were chosen as centroids of different components after all shallower centroids had been removed. Deleting deeper vertices first may add edges among their neighbors, but it never creates an edge between two vertices that have the same centroid depth. Thus each layer is legal to delete at once.

Time complexity: $O(n \log n)$, with at most $\lceil \log_2(n + 1) \rceil$ output operations.

Source note. This editorial rewrites the A–M solution sketches from jiangly's Luogu column and uses the official Universal Cup archive for the stage metadata and problem list.